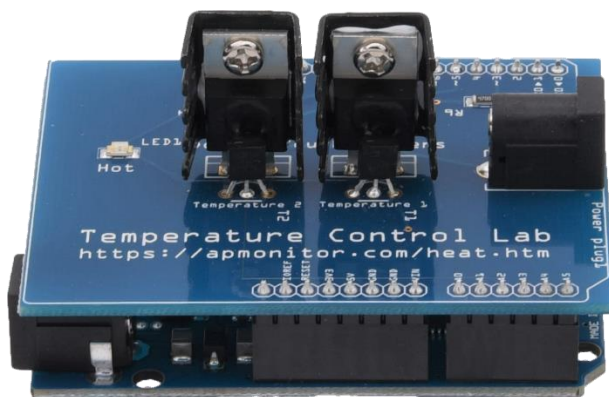


Master in Electrical and Computer Engineering
Master in Aerospace Engineering
2024/2025 – P4

Estimação e Controlo Preditivo Distribuído

Distributed Predictive Control and Estimation

Laboratory Work



Picture source: <https://apmonitor.com/heat.htm>

Prepared by

Afonso Botelho

João Miranda Lemos



Instituto Superior Técnico

Departamento de Engenharia Eletrotécnica e de Computadores

Área Científica de Sistemas, Decisão e Controlo

Ethics statement

When delivering the resolution of this work, the group of students that signs it guarantees, by this act, that the text and all the software and results delivered were entirely carried out by the elements of the group, with a significant participation of all of them, and that no part of the work or software and results presented were obtained from other persons or sources.

Objective

Practice with basic concepts of constrained optimization and receding horizon control (MPC). Perform system identification with data from a real thermal plant with one input and one output, in order to design a model predictive controller (MPC) and Kalman filter in a simulation environment and then apply it back to the real plant.

Pedagogical concept

This laboratory work has been produced in the framework of the pedagogical project MobilLab, financed by IST. According to the pedagogical concept in this project, the students receive a lab kit to perform the experiments according to their own schedule. The kit is then given back to IST at the end of the term. Laboratory classes are used to present progress by the student and to clarify doubts.

Underlying concepts

- Constrained optimization
- Receding horizon control
- System identification
- Model predictive control
- State estimation

Laboratory guide organization

This laboratory guide is organised in five parts:

1. Basics on constrained optimization
2. Receding horizon control
3. Model identification
4. MPC and Kalman filter design
5. Application to real system

Hereafter, P_i identifies part i of the work to be done, as listed above. From P_3 onward, you will be using the TCLab kit, described next.

Kit used

Parts 3 to 5 of this work use the TCLab kit. See the document ***TCLab installation and setup instructions*** in order to connect the kit to your personal computer.

The kit is physically connected to the computer by a USB cable. The kit consists of two connected boards, one with the plant to control and the other an ARDUINO that is used just for communication between the plant board and the personal computer.

The TCLab kit must also be connected to a power supply. See the manual ***TCLab installation and setup instructions*** for instructions.

The plant consists of two power transistors that receive a command signal each between 0 and 100% in order to dissipate heat on metallic plates, and two temperature sensors (thermistors) that provide a temperature reading at each metallic plate. In the present work, we will only use one heater and one sensor.

After interconnecting the TCLab kit with the personal computer, and installing the interface software, it is possible to read each of the temperatures with a single MATLAB command (in degree Celsius), as well as to send commands to each of the transistor heaters (a number between 0 and 100, with 0 corresponding to no heating and 100 corresponding to the maximum possible heating).

A typical MATLAB program to do temperature control consists therefore of a time cycle in which the following activities are done:

1. Read the temperature from the plant (just one command line in MATLAB);
2. Compute the control variable using the predictive control algorithm (this task is the core of the work to be done);
3. Write the value of the control variable to the plant (just one command line in MATLAB);
4. Wait for the sampling time to end and go to step 1.

Safety precautions

Never touch the heating elements (see picture). They can be very hot and injure your skin.

In order not to damage the kit, do not apply high values of the control, such that the temperature is kept below 60 °C.



Software required

To perform this work, the following software is required:

- MATLAB, available through the IST campus license;
- MATLAB Optimization Toolbox, available through the IST campus license;
- MATLAB Control System Toolbox, available through the IST campus license;
- MATLAB System Identification Toolbox, available through the IST campus license;
- TCLab MATLAB interface (function *tclab.m*), available at the Fénix page of the course.
- The following MATLAB scripts: *TCLab_openloop.m*, *TCLab_identification.m*, *TCLab_simulator.m*, *TCLab_closedloop.m* available at the Fénix page of the course.

The information required to perform this work is described below.

Grades

The final grade of the laboratory is computed as

$$L = 0.1 G_1 + 0.2 G_2 + 0.1 G_3 + 0.4 G_4 + 0.2 G_5$$

where G_i is the grade obtained in the part P_i of the work.

Appendix 1 lists the good practices that must be followed when writing the report. Failure to comply with these practices will imply a penalty on the grade. Appendix 1 can be used as a check list to verify the report and improve it.

Participation in laboratory classes

It is not mandatory for the students to perform the work during the laboratory sessions.

Not all the students must be present at all lab classes. At least one student must be present in each class and all the group members must have been present at least once.

Format of the report to deliver

A report is to be delivered, complying with the following format:

- The report can be written in either Portuguese or English. It is advised that you use the language that you master better. Poor language will be penalized.
- File format: pdf
- Each group of students delivers a report, signed by all the group members.
- Include in the 1st page of the report:
 - The name of the curricular unit and the academic year (*ano letivo*)
 - Student number, name and email of all the students in the group
 - An ethical commitment to originality with the following text:
The group of students identified above guarantees that the text of this report and all the software and results delivered were entirely carried out by the elements of the group, with a significant participation of all of them, and that no part of the work or the software and results presented was obtained from other people or sources.
- The answers must be given sequentially from page 2 of the report, indicating at the beginning of each one the respective number (P1, P2, ...) in bold type.
- The maximum number of pages of the report to be delivered, **including the cover page**, is 15, with character font size 12, in LATEX or another word processor.
- Pages must be numbered, with the numbers preferably placed in the centre of the bottom line.

Software to deliver

With the report you must also deliver the following developed code and data:

- The data files for the identification experiments *openloop_data_1.mat* and *openloop_data_2.mat*;
- The identified model data file *singleheater_model.mat*;
- The *mpc_solve* function developed to solve the MPC optimization problem with *quadprog*;
- The modified *TCLab_simulation.m* script configured to perform question 7 of P4;
- The script *TCLab_closedloop.m* and data file *closedloop_data_1.mat* for the closed-loop experiment of P5 in the real plant.

P1 – Basics on constrained optimization

Motivation

The key issue in MPC is that it is based on on-line minimization of a cost function to obtain the value of the control variable to apply to the plant. In this way, MPC allows to enforce constraints on several variables, a feature that is unique among on-line design techniques (optimal control based on Pontryagin's Principle may also embed constraints, but this design technique is an off-line one).

Hence, it is important to get acquainted with algorithms and software for function minimization. This work provides an exercise on the use of popular function minimization MATLAB tools. Although basic, this knowledge is very useful in writing our own tools for MPC, as well as in using MPC toolboxes.

Objectives

Get acquainted to the basics of numerical function minimization using MATLAB Optimization Toolbox through the solution of simple two-dimensional problems, namely

- *fminunc*, for unconstrained function minimization
- *fmincon*, for constrained function minimization

In two-dimensional problems the function to minimize can be represented graphically either by using the MATLAB functions

- *surf*, that draws a 3-dimensional plot of the function
- *contour*, that plots the level curves (the level curves of a function $y = f(x_1, x_2)$ are the curves defined by $f(x_1, x_2) = C$, for different values of the constant C).

Both *surf* and *contour* plot the function in a grid of points that can be defined using the function *meshgrid*.

In addition to the example provided below, you should read the *help* and *doc* (in the MATLAB command window type, for instance, *help fmincon* or *doc fmincon*) to get more information about these functions, as well as the syntax of their use.

A useful example

This example illustrates how to use the minimization software described above to compute unconstrained and constrained minima for a function. This problem is a trivial one, the aim being just to illustrate the use of the MATLAB software.

The function to minimize is

$$f(x_1, x_2) = (x_1 - 1)^2 + (x_2 - 1)^2.$$

It is obvious that this function has an unconstrained minimum for $x = (1, 1)$.

This unconstrained minimization problem is simply formulated as

$$\min_x f(x)$$

To use the MATLAB solver *fminunc* to solve this problem, start by creating a MATLAB function script with the name of the function to minimize. For instance, calling the function BasicFunction, you must create the script:

```
function f = BasicFunction(x)
    f = (x(1)-1)^2 + (x(2)-1)^2;
end
```

To numerically solve the unconstrained optimization problem, define the initial guess of the optimal value of x (called x_0 below) and run the following code

```
options = optimoptions('fminunc','Algorithm','quasi-newton');
xopt = fminunc(@BasicFunction,x0,options)
```

The first line defines the options of the solver (see MATLAB help) and the second line calls *fminunc* to actually solve the problem. The name of the script that contains the definition of the function to minimize is given as the first argument of *fminunc* by the name of the script preceded by the symbol @.

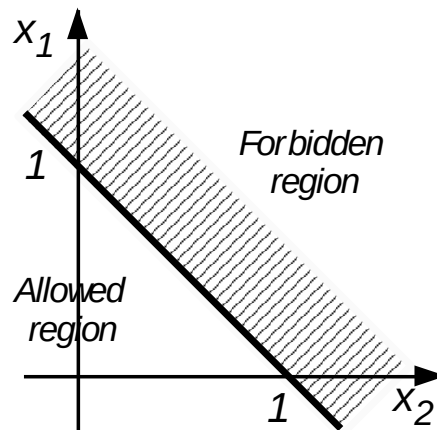


Figure P1-1. Basic optimization example: definition of a linear inequality constraint.

Consider now the introduction of an inequality constraint, such as

$$x_2 \leq 1 - x_1,$$

that corresponds to the allowed values below the straight line in figure P1-1. This constraint can be written in the form

$$Ax \leq B, \text{ with } A = \begin{bmatrix} 1 & 1 \end{bmatrix} \text{ and } B = 1.$$

This form of the constraint is the standard one to be specified in the MATLAB solver software function *fmincon*.

With the above constraint, the optimization problem is formulated as

$$\begin{array}{ll} \min_x & f(x) \\ \text{subject to} & Ax \leq B \end{array}$$

The numerical solution of this constrained problem using *fmincon* is coded in MATLAB by

```
Ac = [1 1];
Bc = 1;
xoptconstr=fmincon(@BasicFunction,x0,Ac,Bc)
```

The first two lines define the constraint. The rest is similar to the use of *fminunc* described above. Of course, one must also define the initial guess of the optimal value of x (called x_0).

Work to perform

0) The MATLAB scripts *BasicFunction.m* and *ProbBasic.m* are available in Fénix. These scripts allow to solve the two unconstrained, and constrained, minimization problems described above. They also plot a 3-D view of the function as well as its level curves. Study and run the script *ProbBasic.m*. You can use it as prototype to the other activities in this set of questions. In this question you don't have to write anything on your report.

1) Compute the unconstrained minimum of the Rosenbrock function, given by

$$f(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2.$$

Compute also the minimum that satisfies the constraint

$$x_1 \leq 0.5.$$

In both cases take as the initial estimate of the minimum

$$x_0 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

In the same plot, show the level lines of the function, the initial estimate (marked with "o"), the unconstrained minimum (marked with "x"), and the constrained minimum (marked with "*"). In another figure, plot the function.

P2 – Basics on receding horizon control

Motivation

The receding horizon strategy is the key component of model predictive control (MPC). It consists of designing the control action such as to optimize it over an extended horizon, apply to the plant only the first control move of the optimized control sequence, and then repeated the process at the next discrete time instant, starting from the new state. It is the receding horizon strategy that allows to transform an open-loop optimal control action into a feedback control action.

When enlarging the optimization horizon, the initial control move approximates the one that would be obtained with an infinite horizon, which is stabilizing.

Objectives

The objective of the work to be done here is to compute the state feedback gain yield by the receding horizon strategy in a problem with linear dynamics, quadratic cost and without constraints, and to compare it with the infinite horizon gain computed using the solution of an equivalent linear quadratic problem with infinite horizon, that involves the solution of an algebraic Riccati equation.

¹ Cost optimization

Infinite horizon LQ problem

Consider the infinite horizon linear quadratic (LQ) optimal control problem that consists of minimizing the infinite horizon unconstrained quadratic cost

$$J_{LQ}(u) = \sum_{t=1}^{\infty} x^T(t)Qx(t) + u^T(t)Ru(t)$$

subject to

$$x(t+1) = Ax(t) + Bu(t).$$

The state x has dimension n and, in general, u has dimension m .

The solution to this problem is obtained by solving, in order to the positive definite $n \times n$ matrix S , the discrete-time algebraic Riccati equation (ARE)

$$A^TSA - S - A^TSB(B^TSB + R)^{-1}B^TSA + Q = 0,$$

and then compute the optimal LQ gain by

$$K_{LQ} = (B^TSB + R)^{-1}B^TSA.$$

The optimal LQ control is given by the linear state feedback

$$u(t) = -K_{LQ}x(t).$$

The optimal LQ control exists if:

1. (A, B) is stabilizable

¹ This symbol means: *study this part carefully at home*.

2. $R \succ 0$ (read $\succ$ positive definite) and $Q \succcurlyeq 0$ (read $\succcurlyeq$ positive semi-definite)
3. (Q, A) has no unobservable mode on the unit circle. If $Q = C^T C$, with C a vector, this condition is verified if (C, A) is observable.

Numerically, the discrete time LQ optimal gain can be computed using the MATLAB Control Systems Toolbox function `d1qr`. The syntax to call this function is

$$[KLQ, S, \lambda] = d1qr(A, B, Q, R)$$

where KLQ is the optimal gain, S is the positive definite solution of the Riccati equation and λ is the vector of eigenvalues of the closed-loop system dynamics $A - BK_{LQ}$.

Finite horizon, receding horizon control

Consider now the finite horizon unconstrained quadratic cost, defined over a horizon with H steps, given by

$$J_{RH}(u; t) = \sum_{i=0}^{H-1} x^T(t+i+1)Qx(t+i+1) + Ru^2(t+i).$$

It is assumed that the plant input is scalar, and hence R is a positive scalar weight.

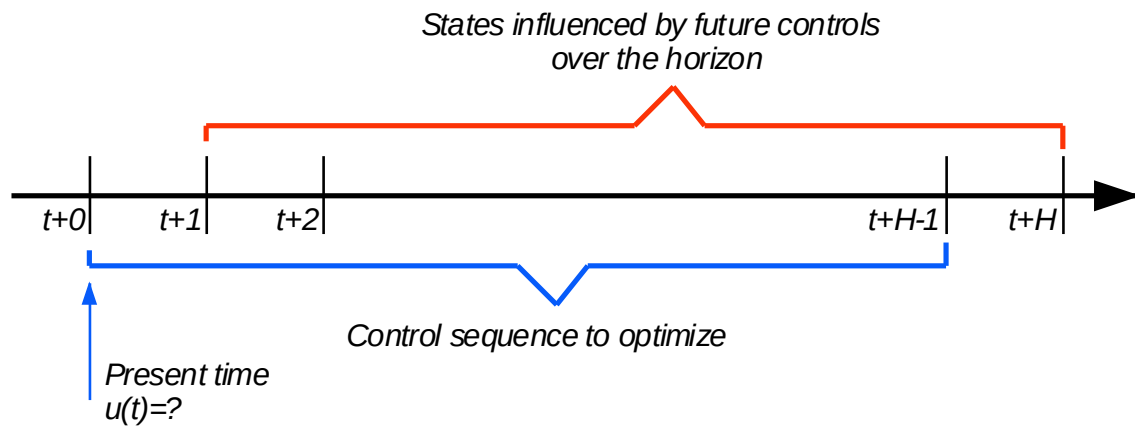


Figure 1 – The control sequence to optimize in the RH cost and the corresponding sequence of states that is influenced by it.

By selecting $Q = C^T C$, where C is a vector in the output equation

$$y(t) = Cx(t),$$

the cost function penalizes the plant output and becomes

$$J_{RH}(u; t) = \sum_{i=0}^{H-1} y^2(t + i + 1) + Ru^2(t + i).$$

Figure P2-1 shows the time synchronization of the variables in this cost function. “We” are at the present time t and want to compute the control sample $u(t)$ to apply to the plant. For that sake, we consider the outputs (or the states) between $t + 1$ and $t + H$. Since the plant has a unit delay between applying a control sample and its influence on the state and the output, this set of future outputs is influenced by the control inputs between t and $t + H - 1$.

The outcome of the optimization is, therefore, the sequence of future control inputs between t and $t + H - 1$. According to the receding horizon strategy, of this whole control sequence, only the first element, $u(t)$ is applied to the plant, the whole procedure being repeated at time $t + 1$ (where the present time will be $t + 1$, the control sequence with respect to which the optimization is performed is $u(t + 1), \dots, u(t + H)$, and the output sequence will be $y(t + 2), \dots, u(t + H + 1)$) and so on.

In order to simplify the notation, it is assumed, without loss of generality, that $t = 0$.

Define the vectors

$$Y = \begin{bmatrix} y(1) \\ \vdots \\ y(H) \end{bmatrix} \quad \text{and} \quad U = \begin{bmatrix} u(0) \\ \vdots \\ u(H - 1) \end{bmatrix}.$$

With these definitions, the receding horizon cost can be written as

$$J_{RH} = Y^T Y + RU^T U.$$

Using the state model, the vector of future output samples, Y , can be expressed as a function of the vector of future control samples, U , and of the initial state, thereby resulting in a function of U . In order to follow this procedure, iterate the state model to compute future values of the state, yielding (remember that time 0 actually corresponds to present time t ; b is used instead of B to emphasize that u is scalar) the following predictive models for the state

$$x(1) = Ax(0) + bu(0),$$

$$x(2) = A^2x(0) + Abu(0) + bu(1),$$

$$x(3) = A^3x(0) + A^2bu(0) + Abu(1) + bu(2),$$

or, in general,

$$x(i) = A^i x(0) + A^{i-1}bu(0) + A^{i-2}bu(1) + \dots + bu(i-1).$$

Predictive models for the output are obtained by multiplying by C , yielding

$$y(i) = CA^i x(0) + CA^{i-1}bu(0) + CA^{i-2}bu(1) + \dots + Cbu(i-1).$$

This set of predictive models, for $i = 1, \dots, H$, can be written in matrix form as

$$\begin{bmatrix} y(1) \\ y(2) \\ y(3) \\ \vdots \\ y(H) \end{bmatrix} = \begin{bmatrix} Cb & 0 & 0 & \dots & 0 \\ CA b & Cb & 0 & \dots & 0 \\ CA^2 b & CA b & Cb & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & 0 \\ CA^{H-1} b & CA^{H-2} b & \dots & CA b & Cb \end{bmatrix} \begin{bmatrix} u(0) \\ u(1) \\ u(2) \\ \vdots \\ u(H-1) \end{bmatrix} + \begin{bmatrix} CA \\ CA^2 \\ CA^3 \\ \vdots \\ CA^H \end{bmatrix} x(0).$$

Define the matrices

$$W := \begin{bmatrix} Cb & 0 & 0 & \dots & 0 \\ CA b & Cb & 0 & \dots & 0 \\ CA^2 b & CA b & Cb & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & 0 \\ CA^{H-1} b & CA^{H-2} b & \dots & CA b & Cb \end{bmatrix}, \quad \Pi := \begin{bmatrix} CA \\ CA^2 \\ CA^3 \\ \vdots \\ CA^H \end{bmatrix}.$$

With these definitions, the pencil of output predictive models can be written as

$$Y = WU + \Pi x(0).$$

Inserting these predictive models in the cost

$$J_{RH} = Y^T Y + RU^T U,$$

this same cost is written as

$$J_{RH} = (U^T W^T + x^T(0) \Pi^T)(WU + \Pi x(0)) + RU^T U$$

or

$$J_{RH} = U^T (W^T W + RI) U + 2x^T(0) \Pi^T W U + x^T(0) \Pi^T \Pi x(0),$$

which is a quadratic form in U . To simplify the notation, define the matrix

$$M := W^T W + RI.$$

With this definition, the finite horizon cost is written

$$J_{RH} = U^T M U + 2x^T(0)\Pi^T W U + x^T(0)\Pi^T \Pi x(0).$$

The gradient of J_{RH} with respect to U , is²

$$\nabla_U J_{RH} = 2U^T M + 2x^T(0)\Pi^T W.$$

Equating the gradient to zero, yields the following equation, satisfied by the optimal control sequence

$$U^T M + x^T(0)\Pi^T W = 0.$$

Transposing, and solving this equation with respect to U , yields the optimal control sequence

$$U^* = -M^{-1}W^T \Pi x(0).$$

According to the receding horizon strategy, only the first entry of this sequence is applied to the plant. Recalling that $x(0)$, actually, corresponds to $x(t)$, and that $u(0)$ corresponds to $u(t)$, the optimal receding horizon control is given by the state feedback

$$u(t) = -K_{RH}x(t),$$

with the optimal feedback gain given by

$$K_{RH} = e_1 M^{-1} W^T \Pi,$$

where

$$e_1 = [1 \quad 0 \quad \dots \quad 0]$$

is a row vector of dimension H .

Work to perform

Consider the open-loop unstable 1st order plant

$$x(t+1) = 1,2 x(t) + u(t).$$

² The gradient with respect to x of $x^T A x$ is $2x^T A$; the gradient with respect to x of $b^T x$ is b^T .

For this plant, develop a MATLAB script to answer the following questions:

- 1) Compute the optimal LQ state feedback gain;
- 2) Compute the optimal receding horizon gain for different values of the horizon H .
Make a plot of the gain as a function of H and superimpose it on a line that corresponds to the optimal LQ gain.
- 3) Make a plot of the absolute value of the closed-loop eigenvalue (the eigenvalue of $A - bK$) and compare it with the stability boundary.
- 4) Discuss the advantage of enlarging the horizon H in relation to the above example. Compare the RH and LQ gains.
- 5) Repeat the study for the open-loop stable 1st order plant

$$x(t + 1) = 0,8 x(t) + u(t).$$

In relation to stability, for which plant is it more advantageous to use enlarged values of the horizon H ?

In the above study, take $Q = 1$ and select values of R that you consider that yield representative results. Try with small and large values of R (you must decide the actual values. The maximum range of H must also be decided by you.

P3 – Model identification

Motivation

MPC requires a dynamic model of the plant to control, in order to perform state predictions. Although it is possible to build a very good model of the TCLab system from first principles, via the laws of heat transfer and radiation, this may not be the case for other systems, due to the high complexity of the system. Therefore, we instead perform system identification, where a plant model is estimated from experimental data.

Furthermore, in order to simplify the resulting MPC optimization problem, the model to identify will be linear. This model will also be used later for the design of a Kalman filter for state estimation, and for building a simulation environment that supports the design of the controller and observer. Since we will only consider a single heater, the identified model will be single-input/single-output (SISO).

Objectives

The objective for this part is the identification of a linear model for the TCLab system from experimental data, considering only one heater and one temperature sensor. Furthermore, we will validate the model using data obtained from a separate experiment.

Linear model identification

We assume that the system can be described by a nonlinear difference equation such as

$$x(k+1) = f(x(k), u(k)),$$

where $x \in \mathbb{R}^n$ is the vector of state variables, $u \in \mathbb{R}^m$ is the vector of control variables, $f: \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n$ is a function that models the system dynamics, and k is the sample index (discrete time). Furthermore, we define the output equation

$$y(k) = g(x(k)),$$

where $y \in \mathbb{R}^p$ is the vector of output variables and $g: \mathbb{R}^n \rightarrow \mathbb{R}^p$ is a function that computes the output from the state variables.

For the TCLab system, the input variables are the two heater controls, and the output variables are the measured temperatures of the two sensors. For this work, however, we will only consider the control of one heater, in which case $m = p = 1$. We assume that we do not know anything about the number of states n , their dynamics f , or output function g . The system is essentially a “black box”, where we only have access to its inputs and outputs. We must therefore obtain a model by exciting the system sufficiently and appropriately.

Assume we want to control the output of the system around a constant equilibrium value $\bar{y}$, to which corresponds the equilibrium state $\bar{x}$, i.e.

$$\bar{y} = g(\bar{x}),$$

and for which there exists a constant control $\bar{u}$ such that the system is kept in equilibrium, i.e.

$$\bar{x} = f(\bar{x}, \bar{u}).$$

We want to find a linearised model of f that holds well in the neighbourhood of $\bar{x}$ and $\bar{u}$, in order to design a controller that can regulate the plant output at $\bar{y}$.

To this end, we can perform an experiment where the system is operated in this neighbourhood, and decompose the commanded inputs u and the measured outputs y into the two components

$$y = \bar{y} + \Delta y,$$

$$u = \bar{u} + \Delta u,$$

where Δy and Δu are the deviations/increments from the equilibrium. We can then estimate the A , B , C and K_e matrices that best explain the observed experimental observations of the Δy and Δu dynamics with the discrete linear time-invariant model

$$\Delta x(k+1) = A\Delta x(k) + B\Delta u(k) + K_e e(k),$$

$$\Delta y(k) = C\Delta x(k) + e(k),$$

where Δx is the deviation from the steady state $\bar{x}$ such that

$$x = \bar{x} + \Delta x,$$

and e is a Gaussian disturbance with a variance that is also estimated.

The controller developed in the next part will therefore consider this linear model for the incremental variables, where we can achieve the desired output $\bar{y}$ by driving Δy to the origin, via a control increment Δu about $\bar{u}$.

Work to perform

The work to perform in this task consists of a series of experiments in which the plant is operated in open-loop, with the control consisting of increments around the equilibrium point.

- 0) Follow the TCLab installation and setup instructions from the guide available in the Fénix course page. In the present work we will not use Simulink, so you may skip the instructions related to that. You do not need to write anything in the report for this question.
- 1) The MATLAB script *TCLab_openloop.m* allows for performing experiments in open-loop, with a pre-defined control applied to the heaters. Modify this script to use a sampling time of 5 seconds; this time interval will be the sampling time of the discrete-time model to identify and of the MPC. Modify the input profile of the first heater such that:
- Initially, the input has a constant value for which the steady-state temperature is between 40 and 50 degrees Celsius. After reaching this equilibrium temperature, hold that input for some time.
 - Afterwards, raise and lower the input by small increments, e.g 5-10%, allowing the temperature to reach equilibrium before changing the input again (the transient duration may be in the order of 1000 seconds). Perform around five such input changes.
 - Change the name of the resulting .mat file to *openloop_data_1.mat*, to be delivered with the final report. Show the plot of the experiment in the report.

A simulated example of the desired profile is shown in Figure 2 for reference. The idea is to drive the system to an equilibrium output $\bar{y}$ for a certain input $\bar{u}$, and apply several step inputs Δu around that equilibrium and observe the output response Δy . We can then estimate the matrices for the linear incremental dynamics that best fit the experimental data.

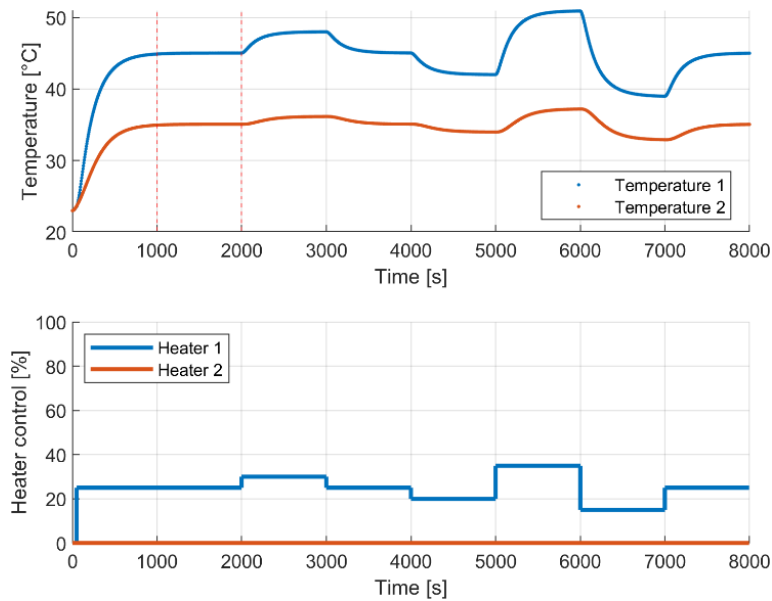


Figure 2 – Simulated example of experiment for model identification.

For the best results, try to minimize energy changes with the environment, e.g. touching and moving the board, opening/closing windows, changing the room air conditioning, etc. You do not need to write anything in the report for this question.

- 2) Perform a second experiment that is shorter in duration (< 1000 seconds), but which excites the system significantly more, with input changes of higher amplitude and without necessarily waiting for the system to reach equilibrium but keeping temperature approximately within the 40–50-degree Celsius interval. Perform about 10 input changes. This second experiment will be used to validate the identified model, given that this data will not be used at all in the identification itself. A simulated example of the desired profile is shown in Figure 3. Change the name of the resulting .mat file to *openloop_data_2.mat*, and include it in the final delivery. For the best results, perform this experiment in the same environmental conditions as that in question 1). You do not need to write anything in the report for this question.

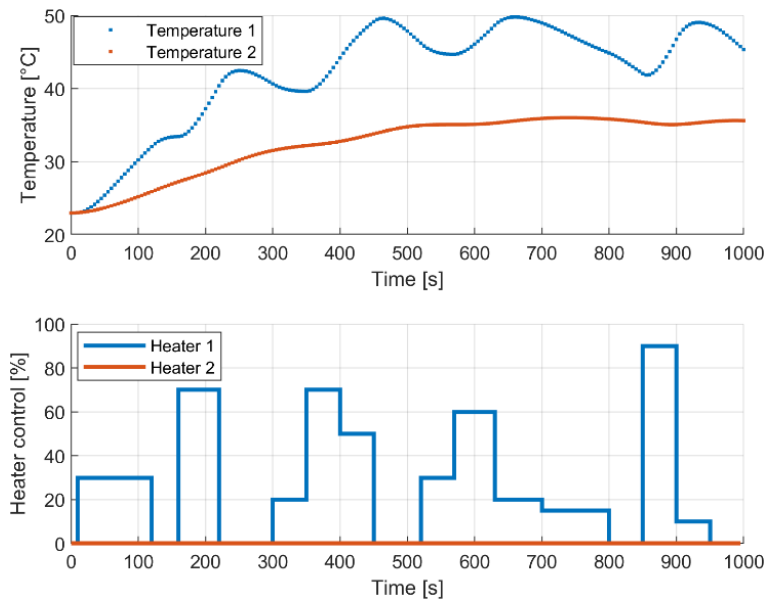


Figure 3 - Simulated example of experiment for model validation.

- 3) Run the script *TCLab_identification.m*. This script performs model identification from the plant data, and has the following functionalities:
 - a. Loads the results of the first experiment and estimates the equilibrium output $\bar{y}$ (y_{ss}) corresponding to the initial input $\bar{u}$ (u_{ss}) by computing the mean value of y during the initial equilibrium interval (the time interval between the two red lines in Figure 2). Change the k_{ss_begin} and k_{ss_end} variables to capture the correct initial and final sample for that time interval.
 - b. Truncates the initial transient and computes the incremental output Δy (Dy) and input Δu (Du).
 - c. Uses the MATLAB *ssest* function to estimate the A , B , C and K_e matrices that best fit the observations, as well as the variance of the disturbance e , with a default state dimension of $n = 1$.
 - d. Estimates the initial state with the *findstates* function and propagates it with the model, plotting the comparison between that result and the experimental data. You should see a very close adherence of the output propagated with the identified model to the experimental output data.
 - e. Loads the data from the second experiment, computes the incremental variables, estimates the initial state and propagates it with the model, again comparing it with the observations. Although the error between

the simulated output and the experimental data should now be greater, the two should still match closely. The MSE between the two is printed in the command window.

Select a suitable model order n taking into account the MSE, while also ensuring that the model does not overfit to the noise, which may happen as the order is increased. Present the plots for both tests in the report for the selected n . For the next parts, we will work with the model generated for that state dimension, which is saved in the *singleheater_model.mat* file.

P4 – MPC and Kalman filter design

Motivation

Given the model previously identified, we will now design a model predictive controller for the system and test it in simulation with the same model, albeit with some disturbances. We will initially consider an unconstrained MPC regulator, and then incrementally complexify the controller to track a non-zero reference and include control and output constraints. Given that the MPC requires knowledge of the state, we will also design a state observer, namely a Kalman filter.

Objectives

The objectives of this part are to design a model predictive controller and a Kalman filter, supported by a simulation environment, before applying them to the real system in the next part.

Constrained MPC – practical aspects

We have seen in P2 that for an unconstrained problem with finite-horizon, quadratic cost and linear dynamics, the optimal receding-horizon control is a simple state feedback, where the feedback gain can be easily determined with algebraic computations from the plant linear model and the cost weights. The real advantage of using a receding-horizon rather than infinite-horizon, however, is that we can directly

include constraints in the problem formulation, such as control limits and safety restrictions. On the other hand, the inclusion of such constraints renders the computation of the solution more complex, generally without a closed-form analytical solution being easily available, as it is in the unconstrained case. In such cases, we need to use a numerical optimization algorithm.

Given that the optimization problem must be solved in real-time, the computation time of the optimizer is the main drawback of this control method. In this case, since the optimization problems considered are quadratic programs, we will use the MATLAB *quadprog* function, which implements an optimization algorithm specific for that class of problem and is therefore more efficient at solving it than the previously used *fmincon* (that must be used, for example, when the cost is not quadratic or the dynamics nonlinear). Naturally, the bigger the problem – number of states, controls, constraints, length of the prediction horizon – the higher the optimizer computational load.

The *quadprog* function allows to minimize a quadratic program with the form

$$\begin{aligned} & \underset{z}{\text{minimize}} && \frac{1}{2} z^T F z + f^T z \\ & \text{subject to} && A_{ineq} z \leq b_{ineq} \\ & && A_{eq} z = b_{eq} \\ & && l_b \leq z \leq u_b \end{aligned}$$

where z is the vector of optimization variables, F is a positive semidefinite matrix, f is a vector, matrix A_{ineq} and vector b_{ineq} define the inequality constraints, matrix A_{eq} and vector b_{eq} define the equality constraints, and l_b and u_b define lower and upper-bounds on z . The function may be called with the syntax

$$z = \text{quadprog}(F, f, A_{ineq}, b_{ineq}, A_{eq}, b_{eq}, l_b, u_b),$$

where any unnecessary inputs may be replaced with an empty variable. To use this function, we must therefore first write our problem in matrix form and determine the parameters F , f , A_{ineq} , b_{ineq} , A_{eq} , b_{eq} , l_b and u_b that correspond to the specific problem at hand.

Without loss of generality, we assume that we are at time $k = 0$, and similarly to that presented in P2 we consider the minimization of the cost function

$$J = \sum_{i=0}^{H-1} \hat{x}^T(i+1)Q\hat{x}(i+1) + R\hat{u}^2(i),$$

where Q is a positive semidefinite matrix and R is a positive definite matrix, subject to the dynamics

$$\hat{x}(i+1) = A\hat{x}(i) + B\hat{u}(i),$$

with the initial state $x(0)$ and output equation

$$\hat{y}(i) = C\hat{x}(i).$$

The ‘hat’ notation in $\hat{x}$, $\hat{u}$ and $\hat{y}$ denotes that these are virtual variables, in the sense that they are predictions made within the MPC, and thus this notation distinguishes them from the real variables x , u and y . From the minimization yields the control profile $(\hat{u}(0), \dots, \hat{u}(H-1))$, out of which we only apply the first, i.e.

$$u(0) = \hat{u}(0),$$

and repeat the process in the next sampling interval.

Similarly to what was previously done in P2, we can define z as the concatenation of all the control variables

$$z = U = \begin{bmatrix} \hat{u}(0) \\ \vdots \\ \hat{u}(H-1) \end{bmatrix},$$

which yields the cost terms

$$F = 2M, \quad f = 2x^T(0)\Pi^TW,$$

where $x(0)$ is the initial state, and M , Π and W are as previously defined in P2. If we do not want to model any constraints, we can simply call the function with

$$U = \text{quadprog}(F, f),$$

and the solution will be the same as that obtained with the method shown in P2.

This is known as the dense formulation, since only the control variables are collocated as optimization variables, yielding a problem that is smaller in size but where the cost

and constraint matrices are denser. This is in contrast to the sparse formulation, where both the state and control variables are collocated, defining instead the optimization vector

$$z = \begin{bmatrix} X \\ U \end{bmatrix},$$

where

$$X = \begin{bmatrix} \hat{x}(0) \\ \vdots \\ \hat{x}(H) \end{bmatrix}.$$

We can write the dynamics in matrix form as

$$\begin{bmatrix} \hat{x}(0) \\ \hat{x}(1) \\ \vdots \\ \hat{x}(H) \end{bmatrix} = \begin{bmatrix} 0 & \cdots & \cdots & 0 \\ A & \cdots & 0 & \vdots \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \cdots & A & 0 \end{bmatrix} \begin{bmatrix} \hat{x}(0) \\ \hat{x}(1) \\ \vdots \\ \hat{x}(H) \end{bmatrix} + \begin{bmatrix} 0 & \cdots & 0 \\ B & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & B \end{bmatrix} \begin{bmatrix} \hat{u}(0) \\ \hat{u}(1) \\ \vdots \\ \hat{u}(H-1) \end{bmatrix} + \begin{bmatrix} I \\ 0 \\ \vdots \\ 0 \end{bmatrix} x(0),$$

Defining the matrices

$$\tilde{A} := \begin{bmatrix} 0 & \cdots & \cdots & 0 \\ A & \cdots & 0 & \vdots \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \cdots & A & 0 \end{bmatrix}, \quad \tilde{B} := \begin{bmatrix} 0 & \cdots & 0 \\ B & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & B \end{bmatrix}, \quad E := \begin{bmatrix} I \\ 0 \\ \vdots \\ 0 \end{bmatrix},$$

we can write

$$X = \tilde{A}X + \tilde{B}U + Ex(0).$$

Note that the 0's in the above matrices may represent zero matrices of appropriate dimension, depending on the state and control dimensions, rather than a scalar 0. In the sparse formulation, we must implicitly include the dynamics as an equality constraint on z , yielding

$$A_{eq}z = b_{eq},$$

where (remember the definition of z given above)

$$A_{eq} := [\tilde{A} - I \quad \tilde{B}], \quad b_{eq} := -Ex(0).$$

Now for the cost function, we can write it in matrix form as

$$J(X, U) = [\hat{x}^T(0) \quad \hat{x}^T(1) \quad \dots \quad \hat{x}^T(H)] \begin{bmatrix} 0 & \dots & \dots & 0 \\ \vdots & Q & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & Q \end{bmatrix} \begin{bmatrix} \hat{x}(0) \\ \hat{x}(1) \\ \vdots \\ \hat{x}(H) \end{bmatrix} \\ + [\hat{u}^T(0) \quad \dots \quad \hat{u}^T(H-1)] \begin{bmatrix} R & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & R \end{bmatrix} \begin{bmatrix} \hat{u}(0) \\ \vdots \\ \hat{u}(H-1) \end{bmatrix}.$$

Defining the augmented cost matrices

$$\tilde{Q} := \begin{bmatrix} 0 & \dots & \dots & 0 \\ \vdots & Q & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & Q \end{bmatrix}, \quad \tilde{R} := \begin{bmatrix} R & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & R \end{bmatrix}$$

yields the quadprog cost matrix

$$F = 2 \begin{bmatrix} \tilde{Q} & 0 \\ 0 & \tilde{R} \end{bmatrix},$$

and $f = 0$.

The sparse and dense formulations are theoretically equivalent, however, depending on the problem and on the optimization algorithm, one may be computationally faster to solve than the other. Furthermore, it is easier in the sparse formulation to define state or output constraints, since the former is directly available, which is not the case for the dense formulation. Let us say we want to constrain the output as

$$G\hat{y}(i) \leq g, \quad i = 1, \dots, H,$$

for some matrix G and vector g , we can write the constraints in matrix form as

$$\underbrace{\begin{bmatrix} G & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & G \end{bmatrix}}_{:=\tilde{G}} \underbrace{\begin{bmatrix} \hat{y}(1) \\ \vdots \\ \hat{y}(H) \end{bmatrix}}_{:=\tilde{y}} \leq \underbrace{\begin{bmatrix} g \\ \vdots \\ g \end{bmatrix}}_{:=\tilde{g}}.$$

For the sparse formulation, we can compute the output vector Y from X with

$$\begin{bmatrix} \hat{y}(1) \\ \vdots \\ \hat{y}(H) \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & C & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & C \end{bmatrix}}_{:=\tilde{C}} \begin{bmatrix} \hat{x}(0) \\ \hat{x}(1) \\ \vdots \\ \hat{x}(H) \end{bmatrix},$$

and the constraints become

$$\tilde{G}\tilde{C}X \leq \tilde{g}.$$

We can therefore implement these constraints with quadprog by defining

$$A_{ineq} = [\tilde{G}\tilde{C} \ 0], \quad b_{ineq} = \tilde{g}.$$

For the dense formulation, the output vector is instead computed from U and $x(0)$ with

$$Y = WU + \Pi x(0).$$

Therefore, the constraints instead become

$$A_{ineq} = \tilde{G}W, \quad b_{ineq} = \tilde{g} - \tilde{G}\Pi x(0).$$

The constraint

$$G\hat{y}(i) \leq g, \quad i = 1, \dots, H$$

Is known as a *hard constraint*, because it must be strictly satisfied by the computed solution. For practical reasons, it may be advantageous to instead consider the relaxed constraint

$$G\hat{y}(i) \leq g + \eta_i, \quad i = 1, \dots, H,$$

where the variables $\eta_i \geq 0, i = 1, \dots, H$ are included as optimization variables, and heavily penalized in the cost function with $\alpha\eta_i^2$ where α is very large compared to the remaining terms. This constraint is known as a *soft constraint*, because although it is strongly encouraged to be satisfied, it is not strictly required. In this case, the optimization vector becomes

$$z = \begin{bmatrix} X \\ U \\ \eta \end{bmatrix},$$

for the sparse formulation, where $\eta = [\eta_1^T, \dots, \eta_H^T]^T$, and

$$z = \begin{bmatrix} U \\ \eta \end{bmatrix},$$

for the dense formulation. The quadprog cost function parameters F and f as well as the constraint parameters A_{eq} , b_{eq} , A_{ineq} , b_{ineq} , l_b and u_b must be changed accordingly to accommodate for the new optimization variables. In particular, the l_b parameter should constrain $\eta_i \geq 0$, $i = 1, \dots, H$, in order to ensure the constraint can only be relaxed, not tightened.

MPC regulator

Let us again consider the single-heater TCLab system. We want to design a model predictive controller that generates an incremental control Δu that drives the incremental output Δy to 0, such that the absolute output y is driven to the equilibrium $\bar{y}$. Assuming without loss of generality that we are at time $k = 0$, we define the following quadratic cost function for the MPC

$$J = \sum_{i=0}^{H-1} \Delta \hat{y}^2(i+1) + R \Delta \hat{u}^2(i),$$

where $R > 0$, which we want to minimize subject to the system model previously identified

$$\Delta \hat{x}(i+1) = A \Delta \hat{x}(i) + B \Delta \hat{u}(i),$$

$$\Delta \hat{y}(i) = C \Delta \hat{x}(i),$$

for $i = 0, \dots, H-1$, and given an initial state $\Delta x(0)$.

Notice that we do not include the disturbance e in the MPC model, since that is a random variable that is obviously unknown *a priori*. Nonetheless, the MPC will reject this disturbance by operating in closed loop, driving the system sufficiently close to the origin despite it.

The absolute control commanded to the plant is therefore $u = \bar{u} + \Delta u$, where Δu is the *feedback* control computed by the MPC, and $\bar{u}$ is a *feedforward* control, which is constant and corresponds to the equilibrium control previously determined in P3. In the absence of perturbations, $\Delta u(k), \Delta y(k) \rightarrow 0$ as $k \rightarrow \infty$, and therefore $u(k) \rightarrow \bar{u}$ and $y(k) \rightarrow \bar{y}$. This control scheme is illustrated in Figure 4.

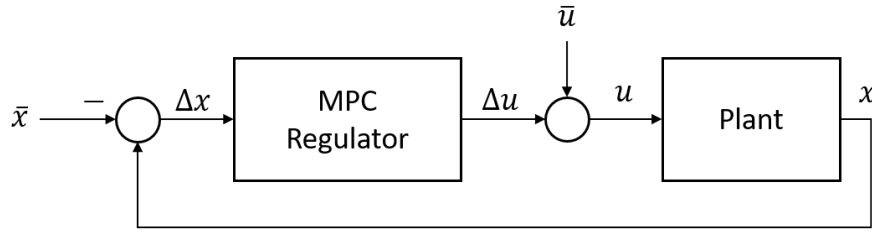


Figure 4 – Illustration of MPC regulator of incremental variables and feedforward control.

Tracking with feed-forward

Let us now assume we instead want to control the output to a reference $r = \bar{y} + \Delta r$. We can design a model predictive controller that tracks the incremental reference Δr , rather than regulating Δy to the origin. One possibility to achieve this is by penalizing the reference tracking error in the cost function

$$J = \sum_{i=0}^{H-1} (\Delta \hat{y}(i+1) - \Delta r)^2 + R \Delta \hat{u}^2(i).$$

This formulation, however, will have a steady-state tracking error unless the system has integral action, due to the fact that tracking Δr requires a non-zero control increment Δu , and therefore the minimization of the tracking error is adversarial to that of the control energy, and the result is something in-between. To overcome this, we can determine the control increment $\Delta \bar{u}$ that maintains the output Δy at the value Δr , and instead formulate the cost function

$$J = \sum_{i=0}^{H-1} (\Delta \hat{y}(i+1) - \Delta r)^2 + R(\Delta \hat{u}(i) - \Delta \bar{u})^2,$$

in which case the control will not be penalized for maintaining Δr . The control $\Delta \bar{u}$ can be determined by solving the steady-state equations

$$\Delta \bar{x} = A \Delta \bar{x} + B \Delta \bar{u},$$

$$\Delta r = C \Delta \bar{x},$$

for $\Delta \bar{x}$ and $\Delta \bar{u}$. This tracking method is essentially a feedforward control, where the MPC computes a control increment about $\Delta \bar{u}$ that regulates the output to the equilibrium Δr .

This is in addition to the feedforward term $\bar{u}$ that we are already using, where the MPC computes a control increment Δu about $\bar{u}$.

We can obtain an equivalent result using the same MPC regulator as before, by first applying the change of variables

$$\delta \hat{x} = \Delta \hat{x} - \Delta \bar{x},$$

$$\delta \hat{y} = \Delta \hat{y} - \Delta r,$$

$$\delta \hat{u} = \Delta \hat{u} - \Delta \bar{u},$$

thus considering $\delta \hat{x}$ and $\delta \hat{u}$ as the new optimization variables instead (or $\delta \hat{u}$ only in case of the dense formulation), which are subject to the model

$$\delta \hat{x}(i+1) = A \delta \hat{x}(i) + B \delta \hat{u}(i),$$

$$\delta \hat{y}(i) = C \delta \hat{x}(i).$$

Naturally, if control and output constraints are present, they should also consider this change of variables.

This control scheme is illustrated in Figure 5.

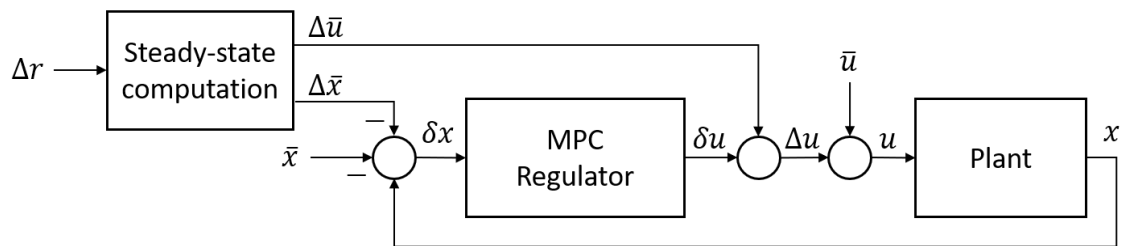


Figure 5 – Illustration of the MPC tracker with feedforward control.

Zero-offset tracking

The previous tracking formulation only guarantees no steady-state error if the real plant matches the MPC model. In practice, however, this will not be the case, e.g. due to changes in the ambient temperature. We can overcome this problem by admitting that the system is affected by an input disturbance d , which results in the dynamics

$$\Delta x(k+1) = A \Delta x(k) + B(\Delta u(k) + d(k)) + K_e \Delta e(k).$$

This changes the steady-state equations to

$$\Delta\bar{x} = A\Delta\bar{x} + B(\Delta\bar{u} + \bar{d}),$$

$$\Delta r = C\Delta\bar{x},$$

which must still be solved for $\Delta\bar{x}$ and $\Delta\bar{u}$, and which requires knowledge of the steady-state disturbance $\bar{d}$. To avoid this assumption, we will estimate the disturbance d together with the state x in the state observer, as shown in the next section, yielding the estimate $\hat{d}$. The best choice for $\bar{d}$ will then be

$$\bar{d} = \hat{d}.$$

Once we have computed $\Delta\bar{x}$ and $\Delta\bar{u}$, we can simply use the same change-of-variable strategy as described in the previous section.

Kalman filter

The model predictive controller requires an initial condition on the state Δx in order to perform the predictions, whereas in reality we only have measurements from the output y . Furthermore, it requires an estimate of the disturbance d in order to achieve zero-offset tracking. Therefore, we need to implement a state observer, which uses the output measurements, command inputs, and a model of the system to estimate the state, which given sufficient time should converge to the real state. For this purpose, we will use a Kalman filter, which designs a linear state observer in an optimal way.

We assume the model

$$\Delta x(k+1) = A\Delta x(k) + B\Delta u(k) + Bd(k) + K_e e(k),$$

$$\Delta y(k) = C\Delta x(k) + e(k),$$

which includes the input disturbance d introduced in the previous section. The Gaussian disturbances e and $K_e e$ have the known covariances

$$E \left[(K_e e(k))(K_e e(k))^T \right] = Q_E,$$

$$E[e(k)e(k)^T] = R_E.$$

Although in this case we have identified these covariances from experimental data, typically this is very hard to do for more complex systems, and the covariances are only interpreted as tuning knobs for the filter, as discussed later.

In order to estimate the disturbance d together with the state x , we will augment the state with the disturbance, yielding the dynamics

$$\begin{bmatrix} \Delta x(k+1) \\ d(k+1) \end{bmatrix} = \begin{bmatrix} A & B \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \Delta x(k) \\ d(k) \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} \Delta u(k),$$

and output equation

$$y(k) = \begin{bmatrix} C & 0 \end{bmatrix} \begin{bmatrix} \Delta x(k) \\ d(k) \end{bmatrix}$$

Defining the augmented state

$$x_d = \begin{bmatrix} \Delta x \\ d \end{bmatrix},$$

the above model can be written as

$$x_d(k+1) = A_d x_d(k) + B_d \Delta u(k),$$

$$\Delta y(k) = C_d x_d(k),$$

where

$$A_d = \begin{bmatrix} A & B \\ 0 & 1 \end{bmatrix}, \quad B_d = \begin{bmatrix} B \\ 0 \end{bmatrix}, \quad C_d = \begin{bmatrix} C & 0 \end{bmatrix}.$$

We also consider the augmented state covariance

$$Q_{Ed} = \begin{bmatrix} Q_E & 0 \\ 0 & \delta_E \end{bmatrix},$$

where δ_E will be a tuning parameter.

The estimation is made in two steps: prediction and correction. At the beginning of the sampling interval k , we first propagate the previous state estimate with the model and the previously commanded control action, i.e. we compute

$$\hat{x}_d^-(k) = A_d \hat{x}_d(k-1) + B_d \Delta u(k-1),$$

which is the prediction step. Afterwards, we correct this estimate using the latest measurement $\Delta y(k)$, thus yielding the final estimate $\hat{x}_d(k)$

$$\hat{x}_d(k) = \hat{x}_d^-(k) + L(\Delta y(k) - C_d \hat{x}_d^-(k)),$$

where L is the observer gain. Note that here we use the ‘hat’ notation to denote an estimate; this should not be confused with the ‘hat’ notation in the context of MPC, which instead denotes a prediction.

The Kalman filter selects the observer gain that minimizes the variance of the estimation error

$$J_E = E \left[\sum_{k=0}^{\infty} \|x_d(k) - \hat{x}_d(k)\|^2 \right],$$

given the disturbance covariances Q_{E_d} and R_E . In practice, we can look at these covariances as tuning knobs for the filter, where decreasing the state disturbance covariance Q_{E_d} relative to the output disturbance variance R_E increases our trust in the model and thus the gain L will be smaller, while increasing it instead increases our trust in the measurements, resulting in a greater gain.

We can compute the Kalman gain L with the MATLAB `dlqe` function, with the syntax

$$L = \text{dlqe}(A_d, \text{eye}(n+1), C_d, Q_{Ed}, R_E),$$

where n is the original state dimension n .

The complete control scheme with the addition of the Kalman filter is presented in Figure 6.

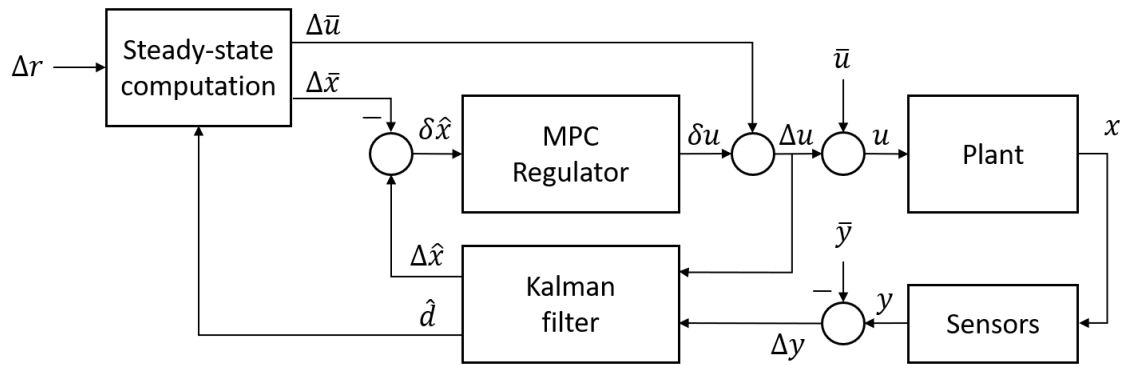


Figure 6 – Illustration of the zero-offset MPC tracker with feedforward control and Kalman filter.

Work to perform

- 0) In this part you will use the *TCLab_simulation.m* MATLAB script provided, which simulates the TCLab system with the model identified in the previous part and an open-loop control profile, starting at the equilibrium temperature for $u = 0$ (ambient temperature). Run the script to test that it runs properly. Notice that variables Dx , Dy , Du correspond to Δx , Δy , Δu respectively, and variables x_{ss} , y_{ss} , u_{ss} correspond to $\bar{x}$, $\bar{y}$, $\bar{u}$ respectively. Furthermore, notice that the simulator uses the same interface as the real system, with function `T1C` to read the temperature, and function `h1` for applying a control action, with the difference that the latter now receives and returns the state, which is not available for the real system. This interface, like the real system, operates in absolute variables rather than incremental ones; however, since the identified model is valid for incremental variables, the constants $c1$ and $c2$, which are the steady-state residuals, are computed such that the incremental model can be used for absolute variables. You may change the simulation duration appropriately in order to answer the following questions, as well as modify the plots as necessary. You do not need to write anything in the report for this question.

Tip: During design, it can be advantageous to momentarily disable the random disturbance e by setting the standard deviation variable `e_std` to 0, in order to test the controller and observer in ideal conditions. You may choose when to use

this disturbance or not for the results presented in the report, but questions 6 and 7 must show results with it on.

- 1) Develop a MATLAB function named `mpc_solve`, in order to solve the unconstrained MPC regulator for a given initial condition using `quadprog`, using either the sparse or dense formulations. Note that a lot of the code developed for solving the receding-horizon problem in P2 can likely be reused. As a sanity check, you may want to apply `mpc_solve` to one of the problems from P2 and ensure that the solution obtained with `quadprog` is the same as before. As a suggestion, your function may have the syntax

`u0 = mpc_solve(x0,H,R,A,B,C),`

where `u0` is first control action, that is actually applied to the plant. This function will be modified in the next sections, and the final version is to be delivered with the report. You do not need to write anything in the report for this question.

- 2) Implement in the *TCLab_simulation* script a closed-loop unconstrained model predictive controller that regulates Δy to 0 using the `mpc_solve` function. Discuss the effect of changing the prediction horizon H and control weight R , and select a suitable horizon H , to be fixed onwards, and adequately justify your choice.
- 3) Include the control limits $[0, 100]$ % as optimization constraints in the model predictive controller, by modifying the `mpc_solve` function appropriately. Choose a value of R , to be fixed onwards, such that the control is saturated at the above limits during the simulation, and show a plot of the result.
- 4) Track a reference Δr of about 5 degrees with the feedforward tracking scheme, by applying a suitable change of variables, computed by solving the steady-state equations, and using the same MPC regulator as before. Plot and show the results. Then, perturb the simulation model constant c_1 with a 10% increase, which simulates changes in the ambient temperature and system identification errors. Plot and discuss the results, and explain in detail why the MPC presents a steady-state tracking error, despite running in closed-loop. Leave this disturbance on for questions 6 and 7 (turn it off for question 5).
- 5) Include a safety constraint for a maximum temperature y of 55 degrees Celsius. Define a reference above this value. Check whether the MPC optimization is

always successful, i.e. that the quadprog output `exitflag` is 1. If not, explain why, and change the constraint to a soft constraint. Plot the results.

- 6) Design a Kalman filter with the `dlqe` function for the augmented state x_d . Use the identified disturbance variance `e_var` to initially select the Q_E and R_E parameters. Implement the filter in the simulator, and test it without the MPC in the loop, instead using the open-loop profile $u = \bar{u}$. Initialize the state estimate by adding an error to the “real” initial state equivalent to an output error of approximately 5 degrees Celsius. Tune the filter parameters if necessary. Show a plot comparing the estimated output $\hat{y}$ and the measured output y , and with the estimated disturbance $\hat{d}$. Discuss the effect of changing the δ_E parameter.
- 7) Add the MPC in the loop again, where it must now receive the state estimate $\Delta\hat{x}$ rather than the “real” state Δx . Compute the feedforward terms $\Delta\bar{x}$ and $\Delta\bar{u}$ using the disturbance estimate $\hat{d}$. Perform a simulation where the reference r commutes to four different values: 50 degrees, 40 degrees, 60 degrees (which cannot be reached due to the safety constraint) and 45 degrees. Allow for equilibrium to be maintained for a few minutes before changing the reference again, to show the steady-state behaviour of the closed-loop system. Show a plot and discuss the results. Discuss how this tracking scheme with the estimation of d and its compensation in the MPC resembles a controller with integral action, e.g. PI controller.

One common rule of thumb when designing a MPC, is that the computation time should be less than 10% of the sampling time, since otherwise it introduces a non-negligible input delay that may negatively affect performance. Although, most likely, this is not a problem in this case, you should ensure that this rule is satisfied for your controller and, if necessary, reduce the prediction horizon, choose a lower-order state dimension, or select the dense/sparse formulation, in order to reduce the computational load.

P5 – Application to the real system

Motivation

In P3 we identified a linearised model for the TCLab system, and in P4 we designed a model predictive controller and Kalman filter for it. Now, we will apply them to the real plant.

Objectives

The objective is to apply the previously developed controller and observer to the real system.

Work to perform

In this section we will work on the script *TCLab_closedloop*, which is essentially a copy of *TCLab_openloop*, with some placeholder suggestions of where to place the Kalman filter and MPC. It should be very straightforward to adapt your code from *TCLab_simulation* to this script.

- 1) Implement the previously designed Kalman filter and MPC in the *TCLab_closedloop* script. It is recommended that you implement the Kalman filter first and test it with an open-loop profile, only afterwards adding the MPC. Perform an experiment with the same specifications as Question 7) of P4 and save the resulting *.mat* file as *closedloop_data_1.mat*, to be delivered with the report. Show a plot and discuss the results. Namely, discuss any differences you may observe between this experiment and the simulation from Question 7) of P4.

References

- [1] Rawlings, James Blake, David Q. Mayne, and Moritz Diehl. *Model predictive control: theory, computation, and design*. Madison, WI: Nob Hill Publishing, 2017. (Available at <https://sites.engineering.ucsb.edu/~jbrow/mpc/>)
- [2] A. Botelho, B. Parreira, P. N. Rosa and J. M. Lemos. *Predictive Control for Spacecraft Rendezvous*. Springer, 2021.



Appendix 1 – Good practice rules to write the report

The lab grades are given based on the quality evaluation of the report, according to the following good practice rules, as well as on the participation of the students on the lab sessions and report elaboration.

1. **Adequacy of the answers** to the questions in the work guide
 - a) The answers must fully respond to the intended
 - b) Conciseness. Use concise text, but that fully expresses the answer
 - c) Clarity. The text and its articulation with the figures included in the report must be clear. If variables not defined in the statement are used, they must be defined in the report.
 - d) Technical correction. There should be neither technical errors in the answers, nor errors in the use of concepts.
 - e) Critical analysis of results. Do the results make sense given what is known and/or when compared to each other? What is the interpretation of the results?
2. **Imagination and originality** in the answers and in the way they are presented.
3. **Correction of the text** and elegance of the style.
 - a) There should be no spelling or syntax errors.
 - b) Preferably use short and clear sentences
 - c) Well articulated text.
 - d) Proper punctuation, including correct punctuation of equations.
4. **Readable and informative graphics and plots**
 - a) Identify the variables that correspond to each axis and, if appropriate, the units in which they are expressed.
 - b) Proper axes font (not too small or too large; type 14 is suggested).
 - c) Use appropriate scales, in order to show the important part of the evolution of the curves and/or the global evolution of the variables, depending on the situation.
 - d) Do not hide important parts of curves with labels.

- e) Do not use a range of variables that is equal to the range of data. In particular, if the variable to plot has periods during which it is constant, do not select the range of the axis to coincide with this constant. For instance, when plotting a square wave between 0 and 1, do not use vertical axes between 0 and 1, but include a small margin, say, axis between -0.1 and 1.1.
- f) Size of the graphics in the document suitable for what you want to show.
- g) Concise graph legends that elucidate the main aspect of the graph.

5. Clear, well-structured and documented programs

1. Correction of programs.
2. Good structure of programs.
3. Adequate and well distributed comments throughout the program.
4. Good program intelligibility.
5. Adequate and short names of variables, explanation of variables with comments.
6. Avoid using *i* and *j* as variables in your program because they are preassigned to the unit imaginary number (type *i* in the MATLAB command window and see the answer; repeat for *j*). You can use *ii* and *jj* as counters, for instance.

– ***End of document*** –

